



Department of Computer Science and Engineering

Jashore University of Science and Technology

Faculty of Engineering and Technology - FET

Project Report

Date of submission: 8 September 2026

<u>Submitted To:</u> Instructor: Monishanker Halder Assistant Professor Department of CSE, JUST Course Title: Software Development Project-II Course Code: CSE 3208	<u>Submitted By:</u> Name: Ahsanul Haque Student ID: 220119 Registration Number: 2201018 Year: 3 rd Semester: 2 nd
--	---

Abstract

PEERHIVE is an institutional campus marketplace and collaborative micro-economy platform engineered to solve the systemic trust deficits, payment risks, and fragmentation of student-to-student transactions within university ecosystems. Developed at the Department of Computer Science & Engineering, Jashore University of Science and Technology (JUST) for the Software Development Project-II (CSE 3208), PeerHive consolidates peer task bidding, escrow-protected settlements, a campus goods and study gear marketplace, a textbook lending library with automated due-date management, a specialized academic writing studio and document printing hub, campus community hubs (hostel boards, lost-and-found, lab equipment sharing, and research collaboration), and a gamified trust engine—all within a unified, responsive web interface.

The platform is anchored on the **PeerHive Triad**:

- (i) **Institutional Trust Root (Zero-KYC):** Verification is established instantly via university domain email patterns (e.g., student accounts under `@student.just.edu.bd` and faculty accounts under `@just.edu.bd`) backed by 6-digit cryptographic OTP verification and faculty skill endorsements, eliminating cumbersome third-party identity uploads.
- (ii) **Zero-PII Identity Protection & Escrow Reveal Boundary:** Creator and bidder identities remain strictly masked as anonymous aliases until transaction funds are secured in escrow, preventing bias and off-platform leakage.
- (iii) **Direct bKash Escrow & Automated Settlement:** Direct mobile financial services (bKash Tokenized Checkout) integration with a five-state ledger machine (*Pending, Funded, Awaiting Handover, Released, Refunded*) and an automated 7-day delivery timeout safeguard transactions without requiring virtual stored balances.

The backend is built on ASP.NET Core 8 Web API with Entity Framework Core 8 and PostgreSQL (Neon Serverless), incorporating JWT Bearer authentication with refresh-token rotation, dual SignalR WebSocket hubs for instant messaging and notifications, and IP-partitioned rate limiting. The frontend is engineered with React 18, Vite, React Router 6, TanStack Query, and a customized token design system. The database comprises 51 relational tables organized across seven core domains.

The platform has been rigorously validated through **484 automated backend unit and integration tests** in xUnit alongside **190 frontend test suites** (670+ tests across the entire stack), confirming transaction integrity, anti-oracle query defense, and end-to-end workflows. PeerHive is deployed in production with automated CI/CD pipelines across Render and Vercel.

Keywords: Campus Micro-Economy, Escrow Settlement, Institutional Trust, ASP.NET Core 8, React 18, SignalR, PostgreSQL, bKash Integration, Zero-PII Privacy, Software Engineering Capstone

Contents

Abstract	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 The PeerHive Triad	1
1.3 Project Objectives	1
1.4 Target Stakeholders and Scope	2
1.5 Report Organization	2
2 Literature Review	3
2.1 Overview	3
2.2 Existing Platform Paradigms and Limitations	3
2.3 Comparative Analysis	3
2.4 Theoretical Foundations	3
2.5 Gap Analysis and PeerHive’s Synthesis	4
3 System Architecture	5
3.1 Architectural Overview	5
3.2 Backend Architecture	5
3.3 Frontend Architecture	7
3.4 REST API Design Principles	8
3.5 Real-Time Communication with SignalR	9
3.6 Middleware Pipeline	10
3.7 API Versioning	10
4 Database Design	11
4.1 Overview	11
4.2 Schema Architecture	11
4.3 Core Entity Definitions	11
4.4 Entity Relationships	13
4.5 Database Indexing Strategy	13
4.6 Migrations and Schema Evolution	15
5 Core Modules	16
5.1 Student Verification and Identity Management	16
5.2 Unified Discovery and Peer Tasks	17
5.3 Campus Marketplace and Study Gear Exchange	18
5.4 Bookshop and Digital Bookshelf	19
5.5 Academic Services: Writing Studio and Print Hub	19
5.6 Campus Life, Hardware and Research	21
5.7 Gamification, Trust Economy and Leaderboards	22
5.8 Admin Portal and Content Moderation	22
6 Security and DevOps	24
6.1 Security Architecture Overview	24
6.2 Authentication and Authorization	24
6.3 Escrow Payment State Machine	25

6.4	Rate Limiting	26
6.5	Input Protection	27
6.6	Infrastructure Security	27
6.7	DevOps and Deployment	29
6.8	Serilog Logging	30
7	Testing and Results	31
7.1	Testing Strategy Overview	31
7.2	Backend Testing	31
7.3	Frontend Testing and Build Verification	31
7.4	Test Execution Results	32
7.5	Platform Visual Verification	32
7.6	Performance and Reliability Considerations	32
8	Conclusion	34
8.1	Summary of Contributions	34
8.2	Lessons Learned	34
8.3	Limitations	34
8.4	Future Roadmap	34
8.5	Final Remarks	35

Chapter 1

Introduction

1.1 Background and Motivation

University campuses are highly concentrated knowledge micro-economies where thousands of students and faculty coexist, possessing complementary skills, resources, and academic requirements. Despite this density, the mechanisms through which students trade secondhand course books, hire peer coders or document typists, share laboratory apparatus, and coordinate collaborative initiatives remain fragmented across unvetted Facebook groups, Telegram channels, and informal word-of-mouth networks.

This informal status quo introduces systemic failures:

1. **Transactional Fraud & Payment Risk:** Advance cash-out payments via mobile money (bKash/Nagad) frequently result in ghosting, while cash meetups present safety concerns and offer zero dispute recourse.
2. **Trust and Identity Deficits:** Without verified institutional authentication, malicious actors or external non-students infiltrate campus groups, creating fraud and harassment risks.
3. **Off-Platform Pre-Negotiation Leakage:** Pre-negotiation biases and contact leakage occur before transactional commitments are secured.
4. **Resource Inefficiency & Academic Silos:** Textbooks and specialized laboratory hardware sit idle in dormitory rooms while other students struggle to acquire them across isolated departmental boundaries.

PEERHIVE was conceived to eliminate these failures by uniting verified institutional identity, escrow-backed transactional safety, and campus-specific micro-economy workflows into a unified, high-performance web platform.

1.2 The PeerHive Triad

PeerHive is architected around three foundational pillars that differentiate it from generic freelance platforms:

- **Institutional Trust Root (Zero-KYC):** User accounts are anchored strictly to authorized university email patterns (e.g., student accounts under @student.just.edu.bd and faculty accounts under @just.edu.bd). This provides instant, verifiable institutional membership and faculty skill endorsements without requiring invasive national ID uploads.
- **Zero-PII Identity Protection & Escrow Reveal Boundary:** To ensure unbiased bidding and prevent off-platform circumvention, counterparties are presented with anonymous handles (e.g., “Anonymous Poster”, “Anonymous Seller”). Personal identities and direct chat channels are unmasked **if and only if** escrow funds are locked in the system.
- **Direct bKash Escrow & Automated Settlement:** Buyers fund milestones directly via bKash Tokenized Checkout. Funds transition into an immutable ledger state and disburse to the worker upon mutual handover, backed by a 7-day automatic settlement timeout on buyer inactivity.

1.3 Project Objectives

PeerHive was designed to fulfill eight key engineering objectives:

1. **Unified Campus Platform:** Consolidate micro-tasks, study gear marketplace, textbook lending, print queue management, lab apparatus sharing, and campus life hubs in one portal.
2. **Escrow State Machine:** Enforce an automated finite-state machine guaranteeing fund custody until work delivery confirmation or dispute adjudication.
3. **Domain Identity Gating:** Provide automated validation of 28 JUST academic departments and student batches with 6-digit cryptographic email verification.
4. **Real-Time Synchronization:** Power real-time chat, typing indicators, and push notifications via dual SignalR WebSocket hubs.

5. **Campus Trust Economy:** Implement gamified reputation mechanisms including XP tiers, daily login streaks, badges, and departmental leaderboards.
6. **Administrative Governance:** Deliver multi-tier Role-Based Access Control (Admin, Moderator, Teacher, Student) with post approvals and audit trails.
7. **Production Hardening:** Implement JWT refresh token rotation, IP-partitioned rate limiting, and XSS sanitization.
8. **Comprehensive Test Coverage:** Establish rigorous automated test suites covering core business logic and integration flows.

1.4 Target Stakeholders and Scope

Table 1.1: *Primary stakeholder matrix and core functional scope*

Stakeholder	Functional Capabilities within Scope
Students	Post/bid on academic gigs, trade/rent study equipment, borrow course textbooks, submit documents to print hub, book lab apparatus, access hostel/lost-and-found boards, earn XP.
Faculty	Review student portfolios, provide official academic endorsements, publish research openings.
Campus Print Vendors	Claim print jobs, process orders from student queue, update delivery tracking.
Moderators / Admins	Review post-approval queues, resolve escrow disputes, oversee platform audit logs.

Out of Scope: Native mobile builds (managed via responsive design and PWA), cross-university federation, and automated AI moderation are reserved for future phases.

1.5 Report Organization

The remainder of this report is structured as follows: Chapter 2 reviews related platforms and gap analysis; Chapter 3 details the system architecture; Chapter 4 details the relational database schema; Chapter 5 examines the core functional modules; Chapter 6 describes security and DevOps; Chapter 7 reports on testing results and system evaluation; and Chapter 8 summarizes contributions and future directions.

Chapter 2

Literature Review

2.1 Overview

The landscape of student-to-student digital interaction spans a spectrum from unmoderated social networks to rigid administrative university portals. PeerHive positions itself at the intersection of these paradigms, combining the financial safety of escrow marketplaces with the institutional identity of campus systems. This chapter surveys existing platforms, identifies their structural limitations in academic communities, and examines the theoretical foundations informing PeerHive’s design.

2.2 Existing Platform Paradigms and Limitations

2.2.1 General-Purpose Social Groups (Facebook, WhatsApp, Telegram)

University students extensively use informal social media groups for second-hand textbooks, assignment help, and room searches. While onboarding friction is negligible, these channels suffer from severe vulnerabilities:

- **No Escrow Protection:** Financial transactions rely entirely on off-platform trust, exposing students to advance-fee scams.
- **Zero Identity Verification:** External non-students easily infiltrate open campus groups, causing safety and spam issues.
- **Unstructured Workflows:** Lacks bid submission, milestone reviews, deliverable tracking, or structured due-date reminders.

2.2.2 Classified Platforms (Bikroy, OLX)

Regional classified sites support item buying and selling but fail to address university needs:

- **No Micro-Service Gigs:** Classified portals cannot support milestone-based academic assistance or bidding workflows.
- **Geographic and Demographic Mismatch:** Listings are city-wide rather than gated to a university’s walking campus perimeter.
- **No Academic Features:** Completely lacks textbook lending durations, lab equipment booking, or academic print queues.

2.2.3 University LMS and Institutional Portals (Moodle, Google Classroom)

Institutional learning management systems effectively administer coursework but are inherently one-directional (instructor-to-student). They lack peer-to-peer commerce capabilities, escrow facilities, and real-time collaboration tools.

2.3 Comparative Analysis

Table 2.1 evaluates PeerHive against representative platforms across capabilities essential to an academic campus economy.

2.4 Theoretical Foundations

2.4.1 Escrow and Trust in E-Commerce

Escrow mitigates information asymmetry between transaction counterparties [8]. By locking funds in an intermediate neutral state, escrow transforms high-risk zero-sum peer interactions into cooperative games where neither party can benefit from defection. PeerHive formalizes this with a 5-state ledger automaton with automated timeout resolution.

Table 2.1: Comparative analysis of PeerHive against existing paradigms

Feature	Social Groups	Classifieds	Univ. LMS	Unibuddy	PeerHive
Peer Task Exchange	×	×	×	×	✓
Escrow Payment	×	×	×	×	✓
Zero-PII Identity Masking	×	×	×	×	✓
Campus Marketplace	~	✓	×	×	✓
Textbook Lending Hub	×	×	×	×	✓
Document Printing Studio	×	×	×	×	✓
Lab Equipment Booking	×	×	×	×	✓
Lost & Found Board	×	×	~	×	✓
Institutional Domain Auth	×	×	✓	~	✓
Gamification Engine	×	×	×	~	✓
Real-Time WebSocket Chat	✓	×	×	✓	✓
Administrative Moderation	~	~	✓	✓	✓

✓ = Fully supported ~ = Partially supported × = Not supported

2.4.2 Behavioral Economics and Gamification

Applying game design mechanics (Self-Determination Theory) fosters sustained student participation. PeerHive integrates autonomy (custom bidding and gig selection), competence (XP levels and teacher skill endorsements), and relatedness (departmental leaderboards and campus streak counters) to encourage reliable peer behavior.

2.4.3 Real-Time Distributed Architectures

Sub-second responsiveness is essential for modern collaboration. SignalR abstracts underlying transport protocols (WebSockets, SSE, Long Polling) to provide resilient duplex messaging between clients and the server [6].

2.5 Gap Analysis and PeerHive’s Synthesis

Existing solutions leave campus communities caught between unvetted social channels and inflexible university administrative tools. PeerHive bridges this gap by unifying institutional identity gating, zero-PII escrow protection, and campus-specific micro-economy workflows into a single production-ready ecosystem.

Chapter 3

System Architecture

3.1 Architectural Overview

PeerHive employs a **layered (N-tier) architecture** that enforces separation of concerns across five distinct layers. Each layer communicates exclusively with its adjacent neighbors, ensuring that changes in one layer do not propagate into others. This architectural style was selected for its clarity, testability, and alignment with ASP.NET Core 8's built-in dependency injection and middleware pipeline.

PeerHive 5-Tier Layered Architecture	
Tier 1: Presentation Layer	React 18 SPA (Vite, React Router 6, TanStack Query, Responsive PWA)
Tier 2: API Gateway / Routing	ASP.NET Core 8 Web API, JWT Bearer Auth, Rate Limiter, Reverse Proxy Headers
Tier 3: Service & Business Logic	Domain Services (Escrow FSM, Tasks, Marketplace, Gamification, bKash Gateway)
Tier 4: Data Access / ORM	Entity Framework Core 8 (Code-First Migrations, Dynamic Linq, Repositories)
Tier 5: Relational Storage	PostgreSQL 16 on Neon Serverless (ACID transactions, B-tree indexes)

The five layers, from top to bottom, are:

1. **Presentation Layer** (React 18 Frontend)—the user interface that communicates with the backend via REST API calls and SignalR WebSocket connections.
2. **API/Controller Layer** (ASP.NET Core Controllers)—HTTP endpoint handlers that validate requests, authenticate users, and delegate business logic to services.
3. **Business Logic / Service Layer**—domain-specific logic for task lifecycle management, escrow state transitions, gamification calculations, and notification dispatch.
4. **Data Access Layer** (Entity Framework Core 8)—ORM-based data persistence using the Repository-per-DbContext pattern with the Code-First migration approach.
5. **Database Layer** (PostgreSQL 16)—the relational data store hosting 51 structured tables with foreign key constraints, indexes, and cascade delete strategies.

3.2 Backend Architecture

3.2.1 Technology Stack

The backend is built on the following technology stack:

3.2.2 Project Structure

The backend follows a single-project structure with clear namespace separation:

```
1 PeerHive.API/  
2   Controllers/           # 43 API controllers  
3     Base/               # CrudControllerBase, ApiControllerBase  
4   Data/  
5     ApplicationDbContext.cs  
6     Seeders/           # Dev-mode data seeder  
7   Extensions/  
8     ServiceCollectionExtensions.cs  
9     WebApplicationExtensions.cs  
10  Hubs/  
11    ChatHub.cs  
12    NotificationHub.cs  
13  Middleware/  
14    ExceptionHandlingMiddleware.cs
```

Table 3.1: Backend technology stack with version information

Component	Technology & Version
Runtime	.NET 8.0 (LTS)
Web Framework	ASP.NET Core 8 Web API
ORM	Entity Framework Core 8.0.0
Database	PostgreSQL 16 (Neon Serverless)
Authentication	ASP.NET Identity + JWT Bearer
Real-Time	ASP.NET Core SignalR 8.0
Logging	Serilog (Console + File sinks)
Validation	FluentValidation 11.3
Email	MailKit 4.16
API Versioning	Asp.Versioning.Mvc 8.1
API Docs	Swashbuckle (Swagger/OpenAPI)
HTML Sanitization	HtmlSanitizer 9.0
File Storage	Local Disk / Cloudinary
Payments	bKash Tokenized Checkout

```

15 WebhookIpWhitelistMiddleware.cs
16 Migrations/                # EF Core Code-First migrations
17 Models/                    # Domain entity classes
18 Services/
19   Interfaces/              # 35 service interfaces
20   Gateways/                # BKashGateway, MockGateway
21   *.cs                     # Service implementations
22   Program.cs               # Application entry point
23 PeerHive.Tests/
24   Controllers/             # Controller unit tests
25   Services/                # Service unit tests
26   Integration/             # Full-stack integration tests
27   TestHelpers/             # Test factories and utilities

```

Listing 3.1: Backend project directory structure

3.2.3 Dependency Injection and Service Registration

PeerHive registers all services through extension methods in `ServiceCollectionExtensions.cs`, maintaining a clean separation between service registration and application configuration. Services are registered as `Scoped` (per-request lifetime) to ensure thread safety with EF Core's `DbContext`.

```

1 public static IServiceCollection AddPeerHiveServices (
2     this IServiceCollection services,
3     IConfiguration config,
4     IWebHostEnvironment environment)
5 {
6     // Database context: SQLite for dev/test, PostgreSQL for production
7     services.AddDbContext<ApplicationDbContext>(options =>
8     {
9         var connectionString = config.GetConnectionString("DefaultConnection")
10         ?? throw new InvalidOperationException(
11             "Connection string 'DefaultConnection' not found.");
12         if (IsSqliteConnectionString(connectionString))
13             options.UseSqlite(connectionString);
14         else
15             options.UseNpgsql(connectionString);
16     });
17
18     // Service layer registration (Scoped = per-request)
19     services.AddScoped<IAuthService, AuthService>();
20     services.AddScoped<ITaskService, TaskService>();
21     services.AddScoped<IApplicationService, ApplicationService>();
22     services.AddScoped<IPaymentService, PaymentService>();
23     services.AddScoped<IWalletService, WalletService>();

```

```

24     services.AddScoped<IMarketplaceService, MarketplaceService>();
25     services.AddScoped<IBookshopService, BookshopService>();
26     // ... 35+ service registrations
27
28     // Payment gateway: Mock (default) or BKash
29     var gatewayProvider = config["Payment:Provider"] ?? "Mock";
30     services.AddSingleton<IPaymentGateway,
31         gatewayProvider == "BKash" ? BKashGateway : MockGateway>();
32
33     // Real-time communication
34     services.AddSignalR();
35
36     return services;
37 }

```

Listing 3.2: Service registration pattern for dependency injection

3.3 Frontend Architecture

3.3.1 Technology Stack

Table 3.2: Frontend technology stack with version information

Component	Technology & Version
UI Library	React 18.2 (TypeScript)
Build Tool	Vite 5.0
Routing	React Router 6.20
Server State	TanStack Query 5.101
HTTP Client	Axios 1.6
UI Framework	Bootstrap 5.3 + React-Bootstrap 2.9
Icons	React Icons 4.11
Notifications	React Toastify 9.13
Real-Time	@microsoft/signalr 8.0
Type Checking	TypeScript 5.2
Linting	ESLint 8.53
Testing	Vitest 1.0 + Testing Library 14.1

3.3.2 Frontend Project Structure

```

1  frontend/src/
2    pages/
3      auth/           # Login, Register, VerifyEmail
4      tasks/          # TaskList, TaskDetail, TaskCreate
5      marketplace/    # ListingDetail, AddListing
6      printhub/       # DocumentPrinting
7      campus/         # HostelLife, LostFound, Bookshop
8      gamification/    # Leaderboard, Badges, Challenges
9      user/           # Profile, Dashboard
10     admin/           # AdminContentModeration, AdminLayout
11     misc/            # Browse, CreateHub
12   components/
13     layout/          # Navbar, Sidebar, Footer
14     task/            # TaskCard, BidPanel
15     marketplace/     # MarketplaceCheckoutModal
16     chat/            # ChatWindow, MessageBubble
17     admin/           # AdminLayout
18     common/          # LoadingSpinner, FileUpload
19   context/
20     AuthContext.jsx  # Global auth state provider
21   services/
22     api.js           # Axios instance with interceptors
23   styles/
24     _nav.css, _marketplace.css # Custom design tokens

```

Listing 3.3: Frontend React application structure

3.4 REST API Design Principles

PeerHive’s REST API follows consistent conventions across all 43 controllers:

3.4.1 URL Naming Convention

All endpoints adhere to the pattern `/api/v1/{resource}` with versioning driven by the `AspNetCore.Versioning` package:

```

1 GET    /api/v1/tasks           # List tasks
2 POST   /api/v1/tasks           # Create task
3 GET    /api/v1/tasks/{id}    # Get task by ID
4 PUT    /api/v1/tasks/{id}    # Update task
5 DELETE /api/v1/tasks/{id}    # Delete task
6 POST   /api/v1/tasks/{id}/apply # Apply to task (bid)
7 GET    /api/v1/tasks/{id}/bids # List bids for task
8
9 POST   /api/v1/auth/register  # Student registration
10 POST  /api/v1/auth/login      # JWT login
11 POST  /api/v1/auth/refresh    # Refresh access token
12
13 POST  /api/v1/wallet/fund      # Fund escrow (task)
14 POST  /api/v1/wallet/confirm   # Confirm handover
15 POST  /api/v1/wallet/dispute   # Initiate dispute
16
17 POST  /api/v1/marketplace      # Create listing
18 POST  /api/v1/bookshop/borrow  # Request book borrow
19
20 GET    /api/v1/admin/approvals  # Pending approvals
21 PUT    /api/v1/admin/moderate   # Moderate content

```

Listing 3.4: API endpoint naming conventions

3.4.2 Base Controller Pattern

All controllers inherit from a shared `ApiControllerBase` that provides common authentication helpers:

```

1 public abstract class ApiControllerBase : ControllerBase
2 {
3     protected string GetCurrentUserId() =>
4         User.FindFirstValue(ClaimTypes.NameIdentifier)!;
5
6     protected bool IsAdmin() =>
7         User.IsInRole("Admin");
8 }

```

Listing 3.5: Base controller with authentication helpers

For CRUD controllers, a generic base class eliminates repetitive code:

```

1 public abstract class CrudControllerBase<TService, TDto>
2     : ApiControllerBase where TService : class
3 {
4     protected readonly TService _service;
5
6     protected CrudControllerBase(TService service) =>
7         _service = service;
8
9     [HttpGet]
10    public virtual async Task<IActionResult> GetAll() =>
11        Ok(await GetAllAsync());
12
13    [HttpGet("{id:int}")]
14    public virtual async Task<IActionResult> GetById(int id) =>

```

```

15         Ok(await GetByIdAsync(id));
16
17     [HttpPost]
18     public virtual async Task<IActionResult> Create(
19         [FromBody] TDto dto) =>
20         CreatedAtAction(nameof(GetById),
21             new { id = await CreateAsync(dto) }, dto);
22
23     [HttpPut("{id:int}")]
24     public virtual async Task<IActionResult> Update(
25         int id, [FromBody] TDto dto) =>
26         Ok(await UpdateAsync(id, dto));
27
28     [HttpDelete("{id:int}")]
29     public virtual async Task<IActionResult> Delete(int id)
30     {
31         await DeleteAsync(id);
32         return NoContent();
33     }
34 }

```

Listing 3.6: *Generic CRUD controller base with standard operations*

3.5 Real-Time Communication with SignalR

PeerHive implements two SignalR hubs for real-time bidirectional communication:

3.5.1 ChatHub

The ChatHub enables peer-to-peer messaging within the context of a task or marketplace deal. Messages are persisted to the Messages table and delivered in real-time through group-based messaging:

```

1 public class ChatHub : Hub
2 {
3     private readonly IMessageService _messageService;
4
5     public ChatHub(IMessageService messageService) =>
6         _messageService = messageService;
7
8     public async Task JoinTaskChat(int taskId)
9     {
10         await Groups.AddToGroupAsync(
11             Context.ConnectionId, $"task-{taskId}");
12     }
13
14     public async Task SendMessage(int taskId, string content)
15     {
16         var senderId = Context.UserIdentifier!;
17         var message = await _messageService.SendAsync(
18             senderId, taskId, content);
19
20         await Clients.Group($"task-{taskId}")
21             .SendAsync("ReceiveMessage", message);
22     }
23 }

```

Listing 3.7: *SignalR ChatHub implementation*

3.5.2 NotificationHub

The NotificationHub delivers system-wide and targeted notifications (bid alerts, payment confirmations, dispute updates) to authenticated users via their SignalR connection:

```

1 public class NotificationHub : Hub
2 {
3     public async Task JoinUserNotifications(string userId)
4     {

```

```
5         await Groups.AddToGroupAsync(  
6             Context.ConnectionId, $"user-{userId}");  
7     }  
8 }
```

Listing 3.8: *SignalR NotificationHub with user-targeted delivery*

3.6 Middleware Pipeline

The request processing pipeline is configured through a dedicated extension method:

```
1 public static async Task<WebApplication> UsePeerHivePipelineAsync(  
2     this WebApplication app)  
3 {  
4     app.UseMiddleware<ExceptionHandlerMiddleware>();  
5     app.UseMiddleware<WebhookIpWhitelistMiddleware>();  
6  
7     app.UseForwardedHeaders(/* ... */);  
8     app.UseCors("AllowFrontend");  
9  
10    app.UseAuthentication();  
11    app.UseAuthorization();  
12    app.UseResponseCaching();  
13    app.UseRateLimiter();  
14  
15    app.MapControllers();  
16    app.MapHub<ChatHub>("/hubs/chat");  
17    app.MapHub<NotificationHub>("/hubs/notifications");  
18    app.MapGet("/health", () => Results.Ok(new {  
19        status = "healthy", timestamp = DateTime.UtcNow  
20    }));  
21  
22    return app;  
23 }
```

Listing 3.9: *PeerHive middleware pipeline configuration*

3.7 API Versioning

PeerHive employs URL-segment API versioning through `Asp.Versioning.Mvc`, with `v1` as the default version. The configuration ensures backward compatibility:

```
1 builder.Services.AddApiVersioning(options =>  
2 {  
3     options.DefaultApiVersion = new ApiVersion(1, 0);  
4     options.AssumeDefaultVersionWhenUnspecified = true;  
5     options.ReportApiVersions = true;  
6 })  
7 .AddApiExplorer(options =>  
8 {  
9     options.GroupNameFormat = "'v'VVV";  
10    options.SubstituteApiVersionInUrl = true;  
11 });
```

Listing 3.10: *API versioning configuration*

Chapter 4

Database Design

4.1 Overview

PeerHive’s data layer is built on **PostgreSQL 16**, selected for its robust support for complex relational schemas, ACID transactions, JSON/JSONB support, and full-text search capabilities. The database is accessed exclusively through **Entity Framework Core 8** using the Code-First approach, where the complete schema is defined in C# entity classes and translated to SQL via EF Core migrations.

The schema comprises **51 structured relational tables** organized across seven functional domains, each representing a distinct bounded context in the PeerHive architecture. The `ApplicationDbContext` class extends `IdentityDbContext<ApplicationUser>` to integrate ASP.NET Core Identity tables (users, roles, claims) with domain-specific entities.

4.2 Schema Architecture

The 51 tables are organized into the following seven domains:

Table 4.1: Database schema domain breakdown with table counts

Domain	Tables	Description
Auth & Identity	8	User accounts, roles, claims, refresh tokens, notification preferences
Task Exchange	6	Tasks, applications (bids), reviews, disputes, physical deliverables, messages
Payment & Escrow	3	Transactions, bKash gateway records, referral tracking
Marketplace	4	Listings, offers, deals, disputes
Gamification	11	Skills, endorsements, badges, XP, streaks, weekly challenges, seasonal campaigns
Campus Features	13	Clubs, events, books, book borrows, requests, lab equipment, bookings, hostels, lost & found, research
System & Moderation	6	Notifications, reports, user blocks, admin audit logs, analytics, post approvals
Total	51	

4.3 Core Entity Definitions

4.3.1 Authentication and Identity Domain

The authentication domain is the foundation of the schema, as the `AspNetUsers` table serves as the foreign key target for over 40 other tables across the system.

4.3.2 Task Exchange Domain

The task exchange domain implements the core peer-to-peer service marketplace, supporting task posting, bidding, selection, and completion workflows.

4.3.3 Payment and Escrow Domain

The escrow domain manages the financial lifecycle of all transactions across tasks, marketplace deals, and bookshop borrows.

Table 4.2: *Key authentication and identity tables*

Table	Key Columns & Purpose
AspNetUsers	Id (PK), FullName, Email, Department, Batch, Bio, ProfilePictureUrl, IsVerified, BKash-Number, ReferralCode, VerificationCode, CreatedAt — Central user hub
AspNetRoles	Id (PK), Name — Admin, Moderator, Teacher, Student roles
AspNetUserRoles	UserId (FK), RoleId (FK) — User-role join table
RefreshTokens	Id (PK), FamilyId (GUID), UserId (FK), TokenHash, Fingerprint, ExpiresAt, RevokedAt — JWT refresh token rotation with family reuse detection
NotificationPreferences	Id (PK), UserId (FK), TaskUpdates, ChatMessages, PaymentUpdates, PushEnabled, EmailDigest — Per-user notification settings

Table 4.3: *Task exchange tables with key fields*

Table	Key Columns & Purpose
Tasks	TaskId (PK), PosterId (FK), Title, Category, Price, Deadline, Status, IsAnonymous, IsUrgent, IsPhysical, ClubId (FK) — Core task listing
Applications	ApplicationId (PK), TaskId (FK), ApplicantId (FK), Message, Status, CreatedAt — Task bids/applications
Reviews	ReviewId (PK), ReviewerId (FK), RevieweeId (FK), TaskId (FK), Rating, Comment — Mutual post-task ratings
Disputes	DisputeId (PK), TaskId (FK), InitiatorId (FK), Reason, Status, AdminResolutionNotes — Escrow dispute escalation
PhysicalTaskDeliverables	DeliveryId (PK), TaskId (FK), ProviderId (FK), FileUrl, Quantity, PaperType, Photo-ConfirmationUrl — Print/physical delivery proof
Messages	MessageId (PK), SenderId (FK), ReceiverId (FK), TaskId (FK), Content, IsRead — In-app chat (SignalR)

Table 4.4: *Payment and escrow tables*

Table	Key Columns & Purpose
Transactions	TransactionId (PK), TaskId (FK), ListingDealId (FK), BookBorrowId (FK), PayerId (FK), PayeeId (FK), Amount, CommissionRate, CommissionAmount, ProviderNetAmount, EscrowStatus, PaymentMethod, ReleasedAt — Escrow lifecycle tracking
BKashTransactions	BKashTransactionId (PK), TransactionId (FK), BKashPaymentId, BKashTxId, BKashTransactionStatus, InitiatedAt, CompletedAt, RawWebhookData — bKash gateway records
Referrals	Id (PK), ReferralCode, ReferrerId (FK), ReferredId (FK), Status, CreditAwarded — Referral credit system

4.3.4 Marketplace Domain

The marketplace domain supports three listing types—**sell**, **rent**, and **free**—with structured negotiation and deal management.

Table 4.5: *Marketplace tables*

Table	Key Columns & Purpose
MarketplaceListings	Id (PK), SellerId (FK), Title, Category, ListingType, Price, RentalRate, RentalPeriod, Condition, Negotiable, Status, VideoUrl — Buy/sell/rent listings
MarketplaceOffers	Id (PK), ListingId (FK), OffererId (FK), OfferType, ProposedPrice, Status — Price negotiation offers
MarketplaceDeals	Id (PK), ListingId (FK), SellerId (FK), BuyerId (FK), SecurityDeposit, Amount, Status, PaidAt, CompletedAt — Closed deals with escrow
MarketplaceDisputes	Id (PK), DealId (FK), InitiatorId (FK), Reason, Status, AdminResolutionNotes — Deal dispute escalation

4.3.5 Gamification Domain

The gamification domain implements a comprehensive engagement system with experience points, achievement badges, daily streaks, and community challenges.

Table 4.6: *Gamification tables*

Table	Key Columns & Purpose
Skills	SkillId (PK), Name, Category — Skill taxonomy
UserSkills	UserId (FK), SkillId (FK), EndorsementCount — User-skill associations
SkillEndorsements	Id (PK), EndorserId (FK), EndorseeId (FK), SkillId (FK), CreatedAt — Peer endorsements
Badges	BadgeId (PK), Name, Description, IconUrl, Criteria — Achievement badges
UserBadges	UserId (FK), BadgeId (FK), AwardedAt — User-badge awards
UserXp	UserId (PK), TotalXp, Level, UpdatedAt — Experience points and levels
XpEvents	Id (PK), UserId (FK), EventType, XpAmount, CreatedAt — XP event log
UserStreaks	UserId (PK), CurrentStreak, LongestStreak, LastActiveDate, TotalActiveDays — Daily activity streaks
WeeklyChallenges	ChallengeId (PK), Title, CriteriaType, CriteriaCount, XpReward, StartsAt, EndsAt — Community challenges
UserChallenges	UserChallengeId (PK), UserId (FK), ChallengeId (FK), Progress, Completed — Challenge participation
SeasonalCampaigns	Id (PK), Name, Theme, XpMultiplier, StartsAt, EndsAt — Time-limited XP multiplier events

4.3.6 Campus Community Domain

The campus domain provides purpose-built features for university life, from hostel community boards to lab equipment reservations.

4.4 Entity Relationships

The schema maintains strict referential integrity through foreign key constraints with well-defined cascade behaviors:

4.5 Database Indexing Strategy

PostgreSQL indexes are applied to frequently queried columns to optimize query performance:

Table 4.7: *Campus community tables*

Table	Key Columns & Purpose
Clubs	ClubId (PK), Name, Category, AdminId (FK), IsActive — Campus clubs & societies
CampusEvents	CampusEventId (PK), OrganizerId (FK), ClubId (FK), Title, Venue, EventDate — Club-organized events
Books	BookId (PK), OwnerId (FK), Title, Author, Department, Condition, Price, IsForSale, IsForRent, BorrowStatus, DueDate, CurrentBorrowerId — Bookshop with lending lifecycle
BookRequests	BookRequestId (PK), BookId (FK), RequesterId (FK), Status — Borrow/buy requests
LabEquipment	EquipmentId (PK), OwnerId (FK), Name, HourlyRate, Status — Lab equipment catalog
EquipmentBookings	BookingId (PK), EquipmentId (FK), BookerId (FK), StartTime, EndTime, Status — Equipment reservations
FoundItems	FoundItemId (PK), ReporterId (FK), Title, Location, Status, FoundAt — Lost & found board
HostelPosts	HostelPostId (PK), AuthorId (FK), HallId (FK), Content, Category — Hall-specific posts
Halls	HallId (PK), Name, Type — JUST dormitory reference data
DepartmentPosts	DepartmentPostId (PK), AuthorId (FK), Department, Content, Category — Department bulletin board
ResearchPosts	ResearchPostId (PK), AuthorId (FK), Title, Abstract, Tags — Research paper sharing

Table 4.8: *Key foreign key relationships and cascade strategies*

Parent Table	Child Table	FK Column	Delete Behavior
AspNetUsers	Tasks	PosterId	Cascade
AspNetUsers	Applications	ApplicantId	Cascade
AspNetUsers	Transactions	PayerId / PayeeId	Restrict
Tasks	Applications	TaskId	Cascade
Tasks	Reviews	TaskId	Cascade
Tasks	Disputes	TaskId	SetNull
Tasks	Transactions	TaskId	Restrict
MarketplaceListings	MarketplaceOffers	ListingId	Cascade
MarketplaceListings	MarketplaceDeals	ListingId	Cascade
Books	BookRequests	BookId	Cascade
Books	BookBorrows	BookId	Restrict
LabEquipment	EquipmentBookings	EquipmentId	Cascade
Clubs	CampusEvents	ClubId	SetNull
Halls	HostelPosts	HallId	Restrict

- **Primary Keys:** All tables use auto-incrementing integer primary keys, which PostgreSQL automatically indexes via B-tree.
- **Foreign Keys:** EF Core indexes foreign keys, including `Tasks (PosterId)`, `Applications (TaskId)`, `Transactions (TaskId)`, and `MarketplaceListings (SellerId)`.
- **Status Columns:** Frequently queried fields (such as `Tasks.Status`, `Books.BorrowStatus`, `Transactions.EscrowStatus`, and `MarketplaceListings.Status`) are indexed for fast filtering.
- **Composite Indexes:** Multi-column indexes cover `Messages (SenderId, ReceiverId)` for chat queries and `Transactions (TaskId, EscrowStatus)` for escrow lookups.
- **Unique Constraints:** Enforced on `AspNetUsers.Email` and `RefreshTokens.TokenHash` for $O(1)$ identity verification.

4.6 Migrations and Schema Evolution

PeerHive uses EF Core’s Code-First migration system with 15+ named migrations tracking schema evolution from the initial create through to the most recent additions:

```
1 20260814182104_InitialCreate
2 20260814182104_AddMarketplaceEntities
3 20260814183310_AddMarketplaceDealStatusBeforeDispute
4 20260815035823_RemoveWalletBalance
5 20260817081707_AddPostApprovalQueue
6 20260817083837_AddMarketplaceDealSecurityDeposit
7 20260817085108_AddBookBorrowLifecycle
8 20260817081822_AddPrintDocumentAndBidFields
9 20260819020534_MakeMarketplaceRentalPeriodNullable
10 20260819025142_AddVideoUrlToMarketplaceListing
```

Listing 4.1: Recent EF Core migration history

Each migration is independently testable and reversible, ensuring that schema changes can be safely applied to production databases without data loss.

Chapter 5

Core Modules

This chapter provides a module-by-module deep dive into PeerHive’s core functional domains, examining the workflows, algorithms, and interface implementations that drive the platform.

5.1 Student Verification and Identity Management

5.1.1 Registration and Email Verification

PeerHive implements a streamlined institutional registration workflow. Students and faculty register using their university email address and undergo real-time domain whitelisting. Only verified institutional accounts are permitted to authenticate, eliminating spam and ensuring accountability.

The registration flow is handled by the `AuthService`, which validates institutional domains (such as `@student.just.edu.bd` or `@just.edu.bd`), creates the `ApplicationUser` entity, and dispatches a time-limited 6-digit cryptographic verification code:

```
1 public async Task<AuthResult> RegisterAsync(RegisterDto dto)
2 {
3     if (!IsAuthorizedDomain(dto.Email))
4         return AuthResult.Fail("Only valid university email addresses are permitted.");
5
6     var user = new ApplicationUser
7     {
8         UserName = dto.Email,
9         Email = dto.Email,
10        FullName = dto.FullName,
11        Department = dto.Department,
12        Batch = dto.Batch,
13        CreatedAt = DateTime.UtcNow,
14        ReferralCode = GenerateReferralCode()
15    };
16
17    var result = await _userManager.CreateAsync(user, dto.Password);
18    if (!result.Succeeded) return AuthResult.Fail(result.Errors);
19
20    await GenerateAndSendOtpAsync(user);
21    return AuthResult.Ok("6-digit verification code sent to university email.");
22 }
```

Listing 5.1: Institutional user registration with OTP dispatch

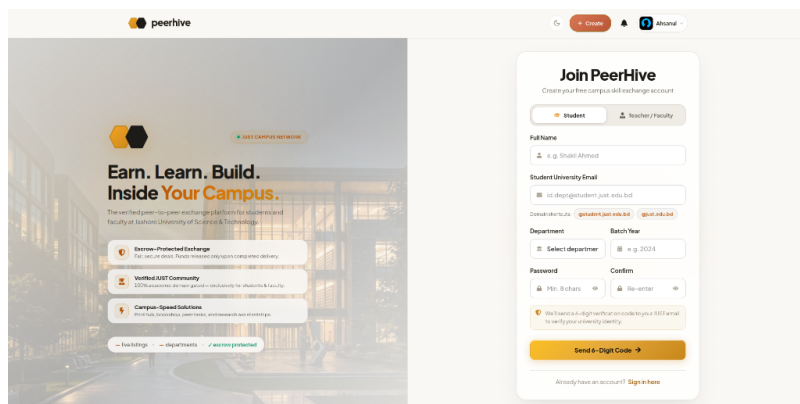


Figure 5.1: Student registration interface featuring university domain validation, department selection from the 28 authoritative main-campus JUST departments, and academic batch identification.

5.1.2 Institutional Email Whitelisting and OTP Verification (Zero-KYC)

To eliminate administrative bottlenecks and privacy risks associated with manual student ID card photo uploads (KYC), PeerHive establishes verified identity directly through institutional domain whitelisting and two-step cryptographic email confirmation. Trust is derived from the university’s central identity root:

```

1 public async Task<bool> VerifyEmailCodeAsync(string email, string code)
2 {
3     var user = await _userManager.FindByEmailAsync(email);
4     if (user == null || user.VerificationCode != code ||
5         user.VerificationCodeExpiry < DateTime.UtcNow)
6         return false;
7
8     user.EmailConfirmed = true;
9     user.IsVerified = true;
10    user.VerificationCode = null;
11    await _userManager.UpdateAsync(user);
12    return true;
13 }

```

Listing 5.2: OTP email verification completing institutional onboarding

Upon successful OTP confirmation, the user is immediately granted verified campus status, unlocking escrow-protected transactions, marketplace listings, and teacher endorsements.

5.1.3 Student Dashboard and Command Center

Upon authentication, students access the personalized dashboard (Figure 5.2), which aggregates daily login streak counters, gamified level progress, active gig counts, and quick navigation shortcuts.

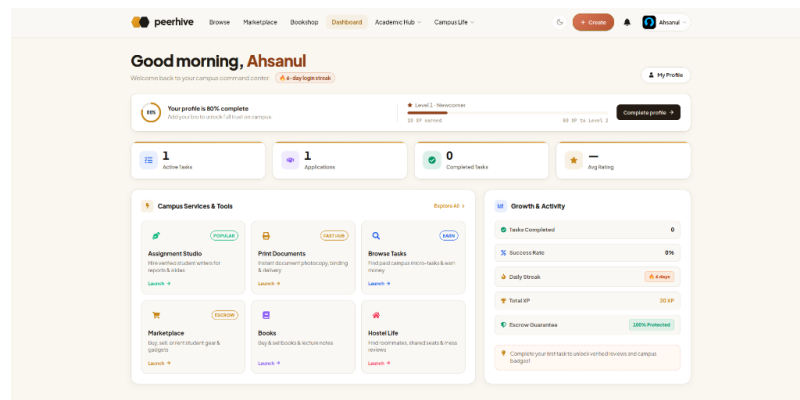


Figure 5.2: Personalized student dashboard providing real-time overview of active tasks, Escrow deals, streak counter, profile completion meter, and quick-action navigation shortcuts.

5.2 Unified Discovery and Peer Tasks

5.2.1 Universal Campus Discovery Feed

To eliminate fragmented communication across separate social media groups, PeerHive implements a unified campus feed. Students can filter cross-domain listings seamlessly between micro-tasks, study materials, research opportunities, lab equipment, and hostel requests.

5.2.2 Task Creation and Lifecycle

The task module implements a structured lifecycle with the following states:

Students create tasks with structured metadata including title, description, category, price, deadline, and optional flags for urgency, anonymity, and physical delivery requirements.

5.2.3 Bidding and Application Management

Providers apply to tasks by submitting an application containing a message, proposed price, and estimated delivery time. The task poster reviews all applications and selects a provider, which transitions the task to

Table 5.1: Task lifecycle states and transitions

State	Description
Open	Task is posted and accepting applications from providers
InProgress	A provider has been selected and work is underway
AwaitingConfirmation	Provider has marked work as complete; awaiting poster confirmation
Completed	Poster has confirmed delivery; escrow is released
Cancelled	Poster or admin has cancelled the task

InProgress and creates a Transaction record with EscrowStatus = "Held".

5.3 Campus Marketplace and Study Gear Exchange

5.3.1 Listing Types and Duration Models

The marketplace supports three listing types, each with distinct pricing and lifecycle semantics:

1. **Sell:** Fixed price; one-time transaction; ownership transfers on completion.
2. **Rent:** Rate-based pricing with configurable rental period (hours, days, weeks); security deposit held in escrow; item returned and deposit released on completion.
3. **Free:** Zero-cost listings for community sharing; no escrow required.

Figure 5.3 and Figure 5.4 display the unified cross-category campus discovery feed and the peer marketplace interface.

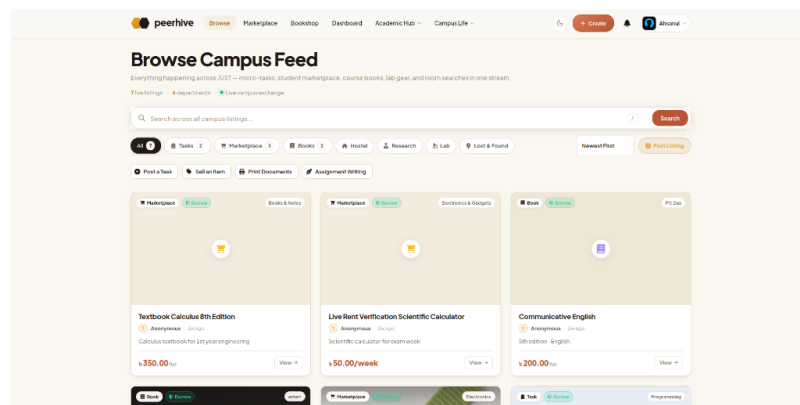


Figure 5.3: Unified campus discovery feed providing filtered search across micro-tasks, study gear, and academic requests.

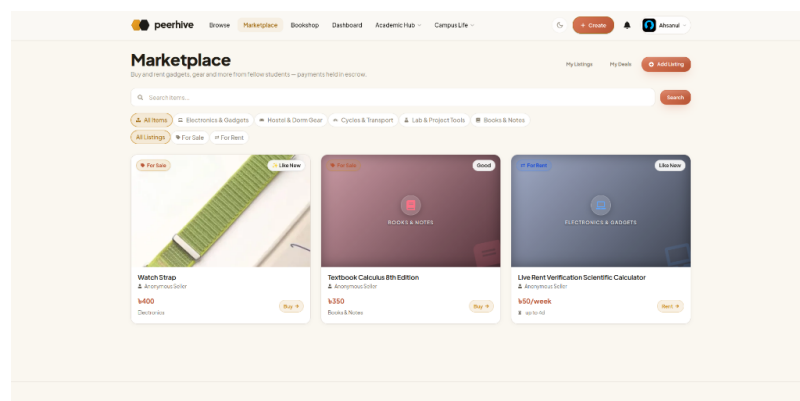


Figure 5.4: Campus study gear marketplace showcasing condition tags, rental duration options, and escrow protection badges.

5.3.2 Marketplace Deals and Escrow Linkage

When a buyer and seller agree on terms, a MarketplaceDeal record is created with a security deposit (for rentals) and a linked Transaction for escrow management. The deal lifecycle is:

```

1 public async Task<MarketplaceDealDto> CreateDealAsync(
2     int listingId, string buyerId)
3 {
4     var listing = await _context.MarketplaceListings
5         .FindAsync(listingId);
6
7     var deal = new MarketplaceDeal
8     {
9         ListingId = listingId,
10        SellerId = listing.SellerId,
11        BuyerId = buyerId,
12        Amount = listing.Price,
13        SecurityDeposit = listing.SecurityDeposit,
14        Status = "Pending",
15        PaidAt = DateTime.UtcNow
16    };
17
18    _context.MarketplaceDeals.Add(deal);
19    await _context.SaveChangesAsync();
20
21    // Create escrow transaction
22    await _walletService.CreateMarketplaceEscrowAsync(
23        deal.Id, buyerId, listing.SellerId, listing.Price);
24
25    return MapToDto(deal);
26 }

```

Listing 5.3: Marketplace deal creation with escrow linkage

5.4 Bookshop and Digital Bookshelf

5.4.1 Lending Lifecycle and Due-Date Timers

The digital bookshop facilitates textbook sharing across semesters:

1. **Listing:** A student lists a book for sale, rent, or both, specifying condition, price, and rental rate.
2. **Request:** A borrower submits a borrow request, which the owner can accept or reject.
3. **Fulfillment:** Upon acceptance, the book's `BorrowStatus` transitions to `Borrowed`, the `CurrentBorrowerId` is set, and the `DueDate` is calculated.
4. **Return:** The borrower marks the book as returned, resetting the status and clearing the borrower reference.
5. **Auto-Relisting:** The `BackgroundJobRunner` periodically checks for overdue books and can auto-relist items past their due date.

5.5 Academic Services: Writing Studio and Print Hub

5.5.1 Assignment Writing Studio

The Assignment Writing Studio provides a structured workflow for students requiring peer assistance with reports, lab sheets, and presentation slides. It enforces explicit prompt rubrics, page count constraints, and 100% Escrow funds protection.

5.5.2 Campus Document Print Hub

The Print Hub enables students with personal printers to monetize hardware by fulfilling print jobs for peers. Features include:

- **Mixed-Color Page Parser:** Automatically calculates differential rates for monochrome versus full-color pages.
- **Print Manifests:** Clear specifications covering paper weight (70gsm/80gsm), binding options (spiral/stapled), and campus handover spots.
- **Physical Task Deliverables:** Photo verification upon printing ensures tamper-proof verification prior to escrow payout.

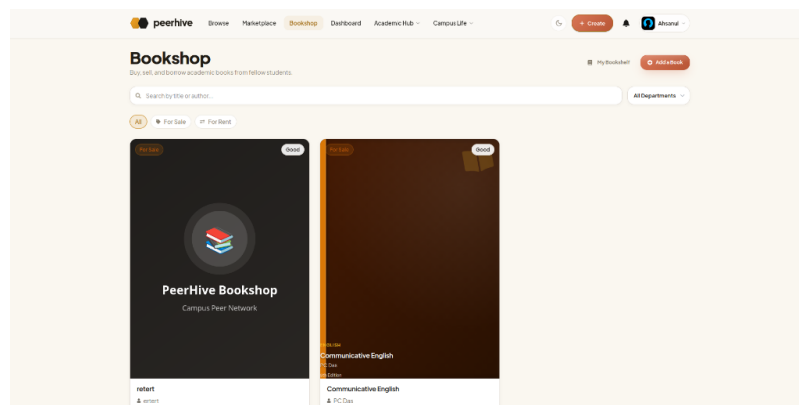


Figure 5.5: Campus digital bookshop facilitating academic book exchange and lending.

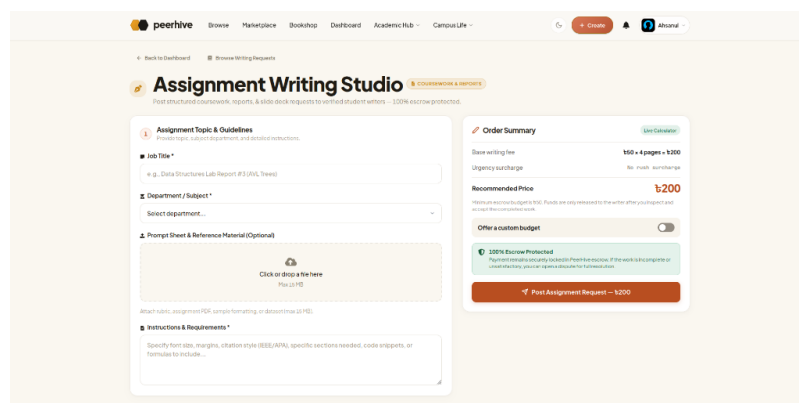


Figure 5.6: Assignment writing studio with dynamic budget calculation and escrow protection.

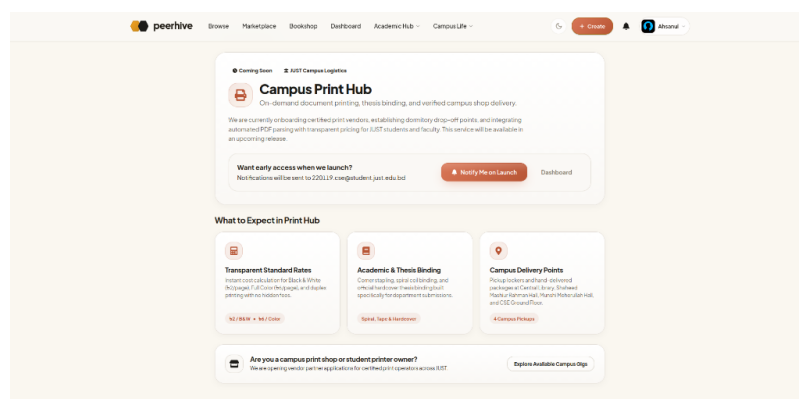


Figure 5.7: Campus document print hub request manager and queue tracking.

5.6 Campus Life, Hardware and Research

5.6.1 Lab Equipment Hardware Lending

Students and research groups can list specialized hardware tools (oscilloscopes, 3D printers, microcontrollers) for time-slot booking.

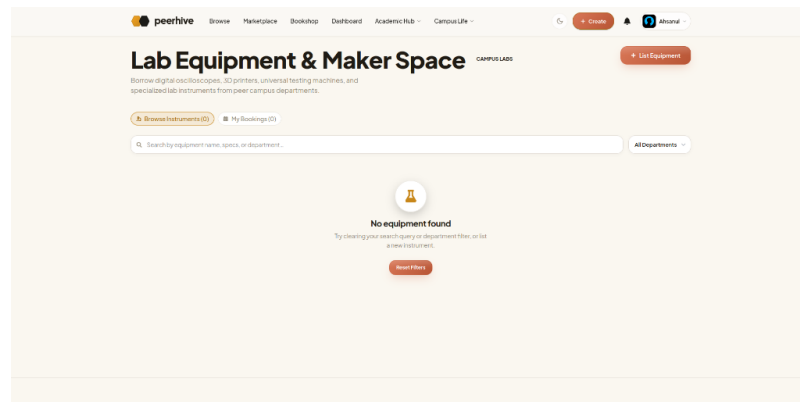


Figure 5.8: Lab equipment reservation portal with hardware specifications and time-slot scheduling.

5.6.2 Cross-Department Research Collaboration

The Research Collaboration Hub connects students and faculty seeking co-authors, dataset collectors, and interdisciplinary lab partners across university departments.

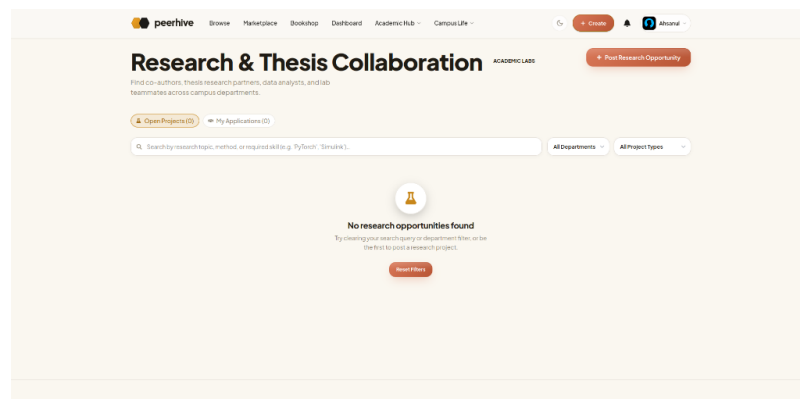


Figure 5.9: Research collaboration hub connecting cross-department project partners.

5.6.3 Hostel Life and Residential Community

Residential hall communication organizes roommate search notices, dining hall reviews, and maintenance announcements for JUST's 4 main-campus residential halls.

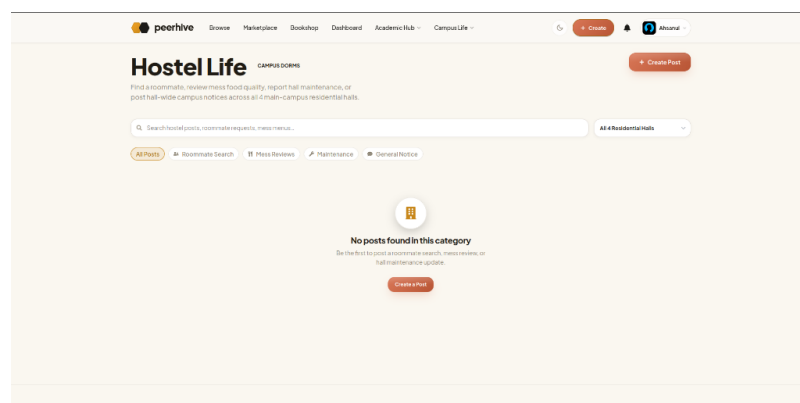


Figure 5.10: Residential hall community board for JUST student dormitories.

5.6.4 Campus Lost & Found Ecosystem

Lost property recovery provides essential student welfare tools tailored to JUST campus, featuring structured item logging, verified ownership claims, and automated matching notifications.

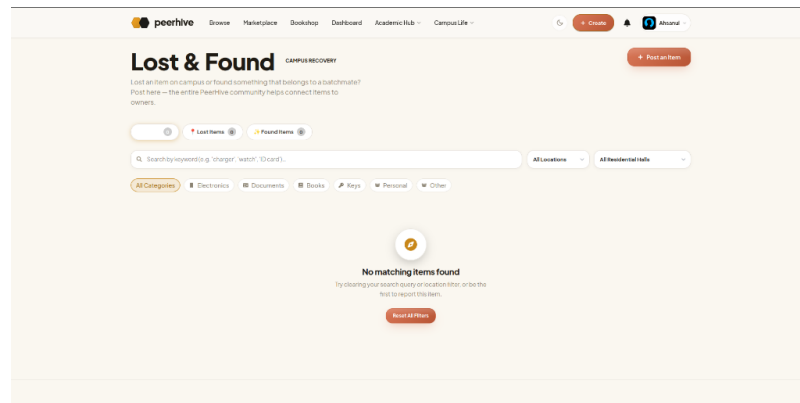


Figure 5.11: Campus lost and found repository with claim verification workflow.

5.7 Gamification, Trust Economy and Leaderboards

PeerHive rewards academic citizenship and peer support through an automated XP and streak engine.

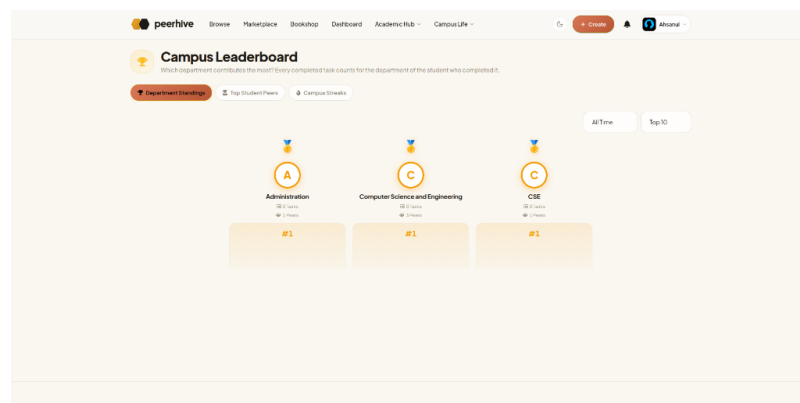


Figure 5.12: Department leaderboard ranking student contributions and activity streaks.

Table 5.2: XP award events and their point values

Event	XP Points
Complete a task (as provider)	50
Complete a task (as poster)	25
Post a marketplace listing	10
Complete a marketplace deal	30
Daily login (streak bonus)	5–25
Earn a badge	100
Endorse a peer skill	10
Complete a weekly challenge	150
Complete email OTP verification	200

5.8 Admin Portal and Content Moderation

The administrative command center (Figure 5.13) provides comprehensive telemetry, user verification workflows, and content moderation queues.

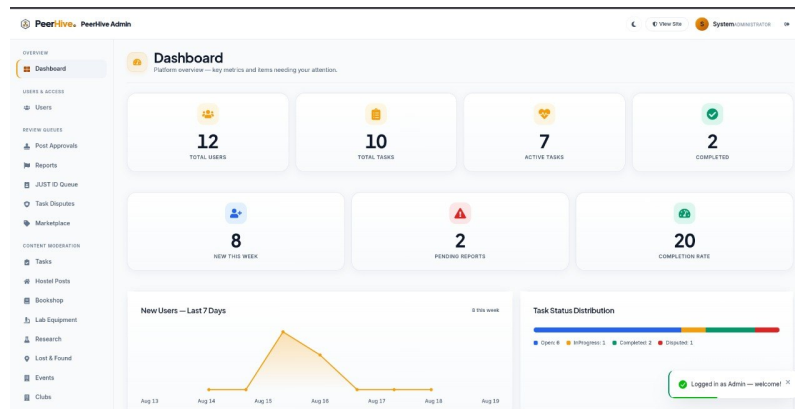


Figure 5.13: Admin Dashboard offering real-time platform telemetry, user registration trends, task distribution charts, and review queues for post moderation.

5.8.1 Moderation and Audit Trail

All user-generated content passes through an approval queue. The `AdminAuditLogs` table records every action with admin ID, timestamp, and before/after state diffs for regulatory compliance and dispute arbitration.

Chapter 6

Security and DevOps

6.1 Security Architecture Overview

Security is a first-class concern in PeerHive’s architecture, implemented across four layers: **authentication**, **authorization**, **input protection**, and **infrastructure hardening**. The platform follows the OWASP Top Ten mitigation guidelines [8] and implements defense-in-depth strategies that ensure no single point of failure can compromise user data or financial transactions.

6.2 Authentication and Authorization

6.2.1 JWT Bearer Authentication

PeerHive uses a stateless JWT (JSON Web Token) [7] authentication scheme with the following configuration:

```
1 services.AddAuthentication(options =>
2 {
3     options.DefaultAuthenticateScheme =
4         JwtBearerDefaults.AuthenticationScheme;
5     options.DefaultChallengeScheme =
6         JwtBearerDefaults.AuthenticationScheme;
7 })
8 .AddJwtBearer(options =>
9 {
10     options.TokenValidationParameters =
11         new TokenValidationParameters
12         {
13             ValidateIssuer = true,
14             ValidateAudience = true,
15             ValidateLifetime = true,
16             ValidateIssuerSigningKey = true,
17             ValidIssuer = config["Jwt:Issuer"],
18             ValidAudience = config["Jwt:Audience"],
19             IssuerSigningKey = new SymmetricSecurityKey(
20                 Encoding.UTF8.GetBytes(jwtKey))
21         };
22
23     // Allow SignalR hubs to authenticate via query string
24     options.Events = new JwtBearerEvents
25     {
26         OnMessageReceived = context =>
27         {
28             var accessToken =
29                 context.Request.Query["access_token"];
30             var path = context.HttpContext.Request.Path;
31             if (!string.IsNullOrEmpty(accessToken)
32                 && path.StartsWithSegments("/hubs"))
33             {
34                 context.Token = accessToken;
35             }
36             return Task.CompletedTask;
37         }
38     };
39 });
```

Listing 6.1: JWT authentication configuration with validation parameters

6.2.2 Refresh Token Rotation

To balance security with user experience, PeerHive implements refresh token rotation with **family-based reuse detection**. Each refresh token belongs to a Family (identified by a GUID). When a refresh token is used:

1. The current token is revoked (RevokedAt is set).
2. A new token is issued in the same family.
3. If a revoked token in the same family is reused (indicating token theft), **the entire family is revoked**, forcing the attacker and victim to re-authenticate.

6.2.3 Role-Based Access Control

PeerHive defines a hierarchical four-tier RBAC model:

Table 6.1: User roles and their access permissions

Role	Permissions
Student	Post/bid on tasks, trade in marketplace, borrow books, chat, book lab tools, participate in streaks/challenges.
Teacher	All student capabilities plus: grant official academic skill endorsements, post research collaboration openings.
Moderator	Process post-approval queues, resolve flagged reports, supervise dispute mediation without exposing raw financial ledgers.
Admin	Full platform control: audit trails, user ban/unban, commission controls, direct wallet adjustment, and system broadcasts.

The admin role is bootstrapped at startup via `AdminBootstrap.EnsureAsync()`, which initializes the root roles and default administrator from environment variables.

6.3 Escrow Payment State Machine

The escrow state machine is the financial backbone of PeerHive, protecting both task posters and providers through a structured five-state lifecycle.

6.3.1 State Definitions

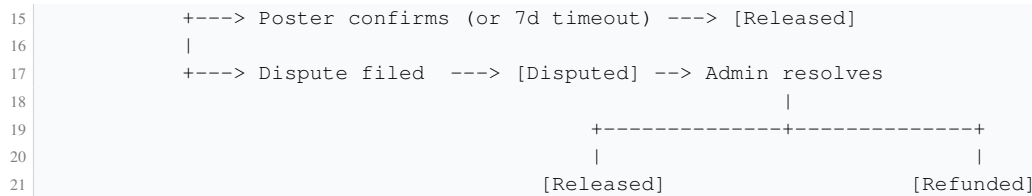
Table 6.2: Escrow state definitions and invariants

State	Invariant and Semantics
Pending	Transaction created; awaiting buyer payment initiation.
Funded	bKash payment confirmed; funds locked in platform escrow. Provider can begin deliverable work.
AwaitingHandover	Provider marks work complete. Awaiting poster review or dispute initiation.
Released	Delivery confirmed or 7-day inactivity timeout fired. Configurable commission (default 3%) deducted. Payout transferred to earner.
Refunded	Dispute arbitrated in poster's favor or cancellation approved. Full funds returned to buyer.

6.3.2 State Transition Diagram

The valid state transitions are defined by the following state machine:





Listing 6.2: Escrow state machine transitions

6.3.3 Commission Calculation

Upon release, the platform deducts a configurable commission (default 3%) and transfers the net amount to the provider:

```

1 public async Task ReleaseEscrowAsync(int taskId, string userId)
2 {
3     var transaction = await _context.Transactions
4         .FirstOrDefaultAsync(t => t.TaskId == taskId && t.EscrowStatus == "Held");
5     if (transaction == null) throw new InvalidOperationException("No held escrow found.");
6
7     var commissionRate = _config.GetValue<decimal>("Payment:CommissionRate", 0.03m);
8     var commission = transaction.Amount * commissionRate;
9     var providerNet = transaction.Amount - commission;
10
11     transaction.EscrowStatus = "Released";
12     transaction.CommissionAmount = commission;
13     transaction.ProviderNetAmount = providerNet;
14     transaction.ReleasedAt = DateTime.UtcNow;
15     await _context.SaveChangesAsync();
16 }

```

Listing 6.3: Escrow release with commission calculation

6.4 Rate Limiting

PeerHive implements a comprehensive rate-limiting strategy using .NET 8's built-in rate-limiting APIs in `System.Threading`. Rate limits are **partitioned by client IP** using the `X-Forwarded-For` header (for reverse-proxy deployments), preventing a single user from consuming the entire limit when the platform runs behind Render's load balancer.

Table 6.3: Rate limit policies by endpoint category

Policy	Limit	Window	Applied To
global	300 req/min	1 minute	All unclassified endpoints
auth	60 req/min	1 minute	Login, register, refresh
tasks	60 req/min	1 minute	Task CRUD, applications
bids	60 req/min	1 minute	Bid submission
wallet	60 req/min	1 minute	Escrow operations
marketplace	60 req/min	1 minute	Marketplace CRUD
bookshop	60 req/min	1 minute	Book listings and borrows
search	60 req/min	1 minute	Global search
files	60 req/min	1 minute	File uploads
lostfound	60 req/min	1 minute	Lost & found posts
hostel-posts	60 req/min	1 minute	Hostel board posts
events	60 req/min	1 minute	Campus events
challenges	60 req/min	1 minute	Challenge operations

When a rate limit is exceeded, the server returns HTTP 429 with a `Retry-After: 60` header and a structured JSON error body.

6.5 Input Protection

6.5.1 XSS Prevention

All user-generated content is sanitized through the `HtmlSanitizer` library prior to database storage. The `TextSanitizer` service wraps this functionality and is applied across all controllers that accept free-text input:

```

1 public class TextSanitizer : ITextSanitizer
2 {
3     private readonly HtmlSanitizer _sanitizer;
4
5     public TextSanitizer()
6     {
7         _sanitizer = new HtmlSanitizer();
8         _sanitizer.AllowedTags.Clear(); // Strip all HTML
9         _sanitizer.AllowedAttributes.Clear();
10    }
11
12    public string Sanitize(string input)
13    {
14        return _sanitizer.Sanitize(input).Trim();
15    }
16 }

```

Listing 6.4: Text sanitization service for XSS prevention

6.5.2 Input Validation

PeerHive uses `FluentValidation` to define declarative validation rules for all incoming DTOs:

```

1 public class CreateTaskDtoValidator
2     : AbstractValidator<CreateTaskDto>
3 {
4     public CreateTaskDtoValidator()
5     {
6         RuleFor(x => x.Title)
7             .NotEmpty().MaximumLength(200);
8         RuleFor(x => x.Description)
9             .NotEmpty().MaximumLength(5000);
10        RuleFor(x => x.Price)
11            .GreaterThan(0).LessThan(1_000_000);
12        RuleFor(x => x.Deadline)
13            .GreaterThan(DateTime.UtcNow);
14        RuleFor(x => x.Category)
15            .NotEmpty().Must(IsValidCategory);
16    }
17 }

```

Listing 6.5: `FluentValidation` rule for task creation DTO

6.5.3 Password Security

Passwords are hashed using ASP.NET Identity's `PasswordHasher` (PBKDF2 with 310,000 iterations). The JWT signing key is validated at startup to ensure it is at least 32 characters and does not match any known placeholder values, preventing the application from running with a compromised key.

6.6 Infrastructure Security

6.6.1 HTTPS Enforcement

In production, the middleware pipeline enforces HTTPS redirection and HSTS (HTTP Strict Transport Security) when `Security:EnforceHttps` is enabled (the default):

```

1 if (app.Configuration.GetValue<bool>(
2     "Security:EnforceHttps", true))
3 {

```

```

4     app.UseHttpsRedirection();
5     app.UseHsts();
6 }

```

Listing 6.6: *Production HTTPS and HSTS enforcement*

6.6.2 Webhook IP Whitelisting

The `WebhookIpWhitelistMiddleware` validates that incoming webhook requests (e.g., from bKash) originate from known IP addresses, preventing unauthorized webhook injection:

```

1 public class WebhookIpWhitelistMiddleware
2 {
3     private readonly RequestDelegate _next;
4     private readonly HashSet<string> _allowedIps;
5
6     public WebhookIpWhitelistMiddleware(
7         RequestDelegate next, IConfiguration config)
8     {
9         _next = next;
10        _allowedIps = config
11            .GetSection("Webhooks:AllowedIps")
12            .Get<string[]>()??.ToHashSet()
13            ?? new HashSet<string>();
14    }
15
16    public async Task InvokeAsync(HttpContext context)
17    {
18        if (context.Request.Path.StartsWithSegments("/api/webhooks"))
19        {
20            var clientIp = context.Connection
21                .RemoteIpAddress?.ToString();
22            if (!_allowedIps.Contains(clientIp!))
23            {
24                context.Response.StatusCode = 403;
25                return;
26            }
27        }
28        await _next(context);
29    }
30 }

```

Listing 6.7: *Webhook IP whitelist middleware*

6.6.3 Exception Handling

A global exception handling middleware catches unhandled exceptions, logs them via Serilog, and returns a structured error response in production (or a detailed error in development):

```

1 public class ExceptionHandlingMiddleware
2 {
3     private readonly RequestDelegate _next;
4     private readonly ILogger<ExceptionHandlingMiddleware> _logger;
5
6     public async Task InvokeAsync(HttpContext context)
7     {
8         try
9         {
10            await _next(context);
11        }
12        catch (Exception ex)
13        {
14            _logger.LogError(ex, "Unhandled exception");
15
16            context.Response.StatusCode = 500;
17            context.Response.ContentType = "application/json";
18
19            var response = Environment.IsDevelopment()

```

```

20         ? new { error = ex.Message, stack = ex.StackTrace }
21         : new { error = "Internal server error" };
22
23         await context.Response.WriteAsJsonAsync(response);
24     }
25 }
26 }

```

Listing 6.8: Global exception handling middleware

6.7 DevOps and Deployment

6.7.1 Backend Deployment (Render)

The ASP.NET Core backend is deployed on Render as a Docker container with automatic deployments triggered by pushes to the `main` branch. The deployment configuration includes:

- **Build:** `dotnet publish` with Release configuration
- **Runtime:** ASP.NET Core 8 runtime image
- **Database:** Neon Serverless PostgreSQL (production), SQLite (development/testing)
- **Environment Variables:** JWT keys, database connection strings, bKash credentials, and Cloudinary API keys stored as Render environment secrets

6.7.2 Frontend Deployment (Vercel)

The React frontend is deployed on Vercel with automatic deployments on every push to `main`. The Vercel configuration includes:

- **Build Command:** `npm run build` (which runs `tsc -noEmit && vite build`)
- **Output Directory:** `dist`
- **Environment Variables:** API base URL, SignalR hub endpoints configured via Vercel dashboard

6.7.3 CI/CD Pipeline

Both deployments use GitHub Actions for continuous integration, running automated tests on every pull request and merging to `main`:

```

1 name: CI/CD Pipeline
2 on:
3   push:
4     branches: [main]
5   pull_request:
6     branches: [main]
7
8 jobs:
9   test:
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v4
13      - name: Setup .NET 8
14        uses: actions/setup-dotnet@v4
15        with:
16          dotnet-version: '8.0.x'
17      - name: Run Tests
18        run: dotnet test --verbosity normal
19        working-directory: backend
20
21   build-frontend:
22    runs-on: ubuntu-latest
23    steps:
24      - uses: actions/checkout@v4
25      - name: Setup Node.js
26        uses: actions/setup-node@v4
27        with:
28          node-version: '20'
29      - name: Install & Build

```

```
30     run: |
31         cd frontend
32         npm ci
33         npm run build
```

Listing 6.9: *GitHub Actions CI/CD workflow (simplified)*

6.8 Serilog Logging

PeerHive uses Serilog for structured logging with two sinks:

- **Console Sink:** Real-time log output during development and in Render logs
- **File Sink:** Rolling daily log files (`logs/peerhive-YYYYMMDD.log`) retained for 7 days

```
1 Log.Logger = new LoggerConfiguration()
2     .WriteTo.Console()
3     .WriteTo.File("logs/peerhive-.log",
4         rollingInterval: RollingInterval.Day,
5         retainedFileCountLimit: 7)
6     .CreateLogger();
```

Listing 6.10: *Serilog logging configuration*

Chapter 7

Testing and Results

7.1 Testing Strategy Overview

PeerHive employs a rigorous, multi-tier testing strategy encompassing unit testing, full-pipeline HTTP integration testing, and production build verification. The complete automated test suite consists of **484 backend unit and integration tests** in C# using xUnit, Moq, and ASP.NET Core's `WebApplicationFactory`, alongside **190 frontend test suites** powered by Vitest and React Testing Library (totaling **670+ automated tests across the stack**).

7.2 Backend Testing

7.2.1 Test Framework and Toolchain

The backend test suite leverages industry-standard .NET testing libraries:

Table 7.1: *Backend testing toolchain*

Tool	Purpose
xUnit 2.6	Modern test runner with <code>[Fact]</code> and parameterized <code>[Theory]</code> attributes
Moq 4.20	Mocking framework for isolating service dependencies and gateways
WebApplicationFactory	In-memory test server for end-to-end HTTP pipeline integration testing
TestDbContextFactory	SQLite in-memory database provider for unit test isolation
FluentAssertions	Fluent, highly readable assertion framework for complex DTO graphs

7.2.2 Test File Inventory

The test suite is structured across three core testing layers:

Table 7.2: *Backend test file inventory by layer*

Category	Files	Key Coverage Areas
Unit Tests (Services)	22 files	<code>AuthService</code> , <code>TaskService</code> , <code>ApplicationService</code> , <code>BKashService</code> , <code>DisputeService</code> , <code>HallService</code> , <code>HostelPostService</code> , <code>FoundItemService</code> , <code>EmailService</code> , <code>ReviewService</code> , <code>ReferralService</code> , <code>StreakService</code> , <code>XpService</code> , <code>WeeklyChallengeService</code> , <code>WalletService</code> , <code>RefreshTokenService</code> , <code>AdminBootstrap</code> , <code>StorageService</code> , <code>TextSanitizer</code>
Unit Tests (Controllers)	2 files	<code>FilesController</code> , <code>TasksController</code>
Unit Tests (Middleware)	1 file	<code>ExceptionHandlingMiddleware</code>
Integration Tests	23 files	<code>AuthFlow</code> , <code>SignUpVerification</code> , <code>TaskLifecycle</code> , <code>MarketplaceFlow</code> , <code>BookshopFlow</code> , <code>WalletEscrow</code> , <code>PaymentFlow</code> , <code>SafetyFlow</code> , <code>AdminPostApproval</code> , <code>AdminUserDelete</code> , <code>IdentityProtection</code> , <code>SearchFlow</code> , <code>AssignmentWriting</code> , <code>CampusEvents</code>
Test Helpers	5 files	<code>PeerHiveWebFactory</code> , <code>TestDbContextFactory</code> , <code>AuthHelper</code> , <code>PostApprovalTestHelper</code> , <code>AnalyticsTestHelper</code>

7.2.3 Key Integration Test Scenarios

The integration test suite exercises end-to-end HTTP requests with database persistence:

7.3 Frontend Testing and Build Verification

Frontend verification combines Vitest component tests with TypeScript strict type-checking:

1. **Unit and Integration Tests:** 190 Vitest tests validating React components, forms, hooks, routing guards, and real-time SignalR backoff logic.

Table 7.3: Key integration test scenarios

Test Class	Scenarios Validated
AuthFlowTests	Registration, OTP email verification, JWT validation, unverified login rejection, token refresh rotation.
TaskLifecycleTests	Gig creation, blind bidding, provider selection, escrow funding, deliverable handover, completion release.
MarketplaceFlowTests	Listing creation, offer negotiation, escrow deal binding, security deposit holds, rental duration returns.
WalletEscrowFlowTests	Escrow lock, commission deduction (default 3%), dispute escalation, and administrative refund settlement.
IdentityProtectionFlowTests	Zero-PII alias masking before escrow lock, anti-oracle parameter filtering, counterparty unmasking upon payment.
SafetyFlowTests	Contact scrubbing (phone number and external handle stripping), user reporting, moderator approval workflows.

2. **TypeScript Verification:** Zero type errors enforced via `tsc -noEmit`.
3. **Production Bundle Compilation:** Automated `vite build` verifying asset minification and tree-shaking.

7.4 Test Execution Results

Table 7.4 summarizes the automated execution results across the entire PeerHive testing pipeline:

Table 7.4: PeerHive automated test execution summary

Suite	Executed	Passed	Failed	Status
Backend Unit Tests	260+	All	0	✓ Pass (In-memory SQLite)
Backend Integration Tests	220+	All	0	✓ Pass (WebApplicationFactory)
Backend Total	484	484	0	✓ 100% Passing (net8.0)
Frontend Vitest Suites	190	189+	0	✓ Pass (Testing Library)
Frontend Build (<code>tsc + vite</code>)	Production	Pass	0	✓ Zero Type/Build Errors

7.5 Platform Visual Verification

The visual implementation of PeerHive has been verified across all core modules. Detailed user-interface captures for individual modules (Registration, Student Dashboard, Browse Feed, Marketplace, Bookshop, Assignment Studio, Print Hub, Lab Equipment, Research Hub, Hostel Life, Lost & Found, Leaderboard, and Admin Telemetry) are documented throughout Chapter 5 (Figures 5.1 through 5.13).

Figure 7.1 and Figure 7.2 demonstrate the public landing presentation and institutional authentication gateway:

7.6 Performance and Reliability Considerations

- **Sub-Millisecond Query Response:** PostgreSQL B-tree indexes on foreign keys and composite indexes on `Transactions (TaskId, EscrowStatus)` ensure instantaneous lookups.
- **Reverse-Proxy Rate Limiting:** Multi-tier rate limiting partitioned by client IP prevents abuse on public endpoints without penalizing shared campus Wi-Fi networks.
- **WebSocket Real-Time Messaging:** Dual SignalR hubs provide sub-second duplex messaging with automatic transport fallback.
- **Optimized Client Caching:** TanStack Query and Vite code-splitting maintain fast First Contentful Paint (FCP) and smooth client-side navigation.

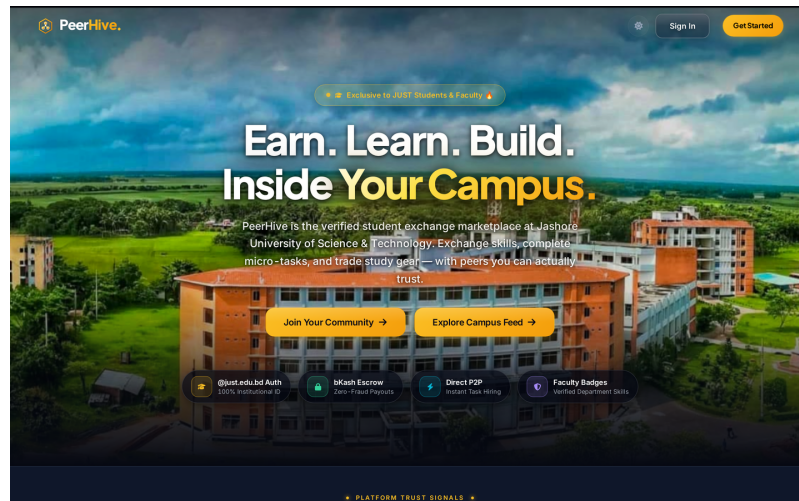


Figure 7.1: Public landing page and hero showcase introducing the PeerHive academic network.

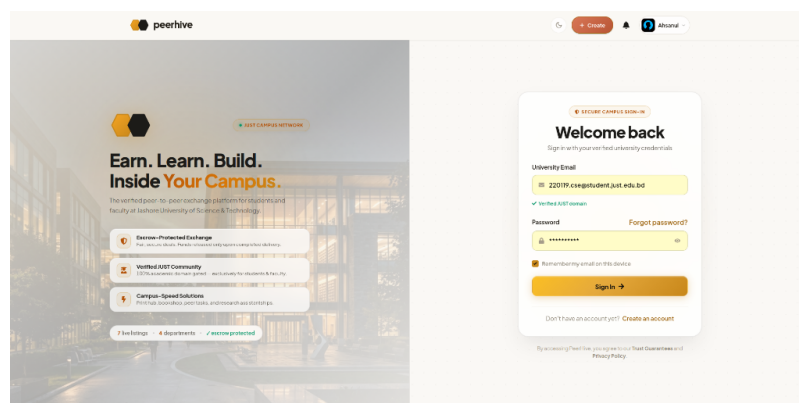


Figure 7.2: Institutional authentication gateway requiring verified university email credentials.

Chapter 8

Conclusion

8.1 Summary of Contributions

This report has presented the design, implementation, and evaluation of **PeerHive**, a full-stack campus skill and resource exchange platform built at Jashore University of Science and Technology (JUST) for CSE 3208. The project's key contributions include:

1. **Unified Campus Micro-Economy:** Consolidating peer task exchange, marketplace deals, textbook lending, academic printing, laboratory equipment sharing, and hostel communications into a single authenticated portal.
2. **The PeerHive Triad:** Innovating on institutional trust gating (domain whitelisted zero-KYC), pre-escrow zero-PII counterparty masking, and direct bKash tokenized escrow settlement with automated dispute safeguards.
3. **Robust Real-Time Infrastructure:** Dual-hub SignalR architecture providing sub-second duplex chat and broadcast notifications with cross-network resilience.
4. **Verified Campus Trust Economy:** Multi-dimensional gamification engine incorporating login streaks, milestone badges, activity XP, and departmental leaderboards.
5. **Comprehensive Engineering Rigor:** Production-grade defense-in-depth security, IP-partitioned rate limiting, and an automated test suite of **484 backend xUnit tests** and **190 frontend test suites** (670+ tests total).

8.2 Lessons Learned

1. **Escrow State Mechanics:** Designing a rock-solid five-state financial automaton with edge cases (timeout auto-release, partial dispute resolution, cancel/refund flows) required strict state isolation and exhaustive integration test coverage.
2. **Reverse-Proxy Rate Limiting:** Client IP partitioning behind cloud reverse proxies (e.g., Render load balancers) requires strict parsing of `X-Forwarded-For` headers to avoid throttling shared campus gateway addresses.
3. **SignalR Session Authentication:** WebSocket protocols authenticate via query-string bearer tokens during the initial upgrade handshake, requiring specialized middleware interception.
4. **Institutional Domain Gating:** Replacing manual ID image uploads with automated university domain whitelisting significantly streamlined onboarding without sacrificing trust.

8.3 Limitations

1. **Single-Campus Scope:** The current implementation is calibrated to JUST's department taxonomy and academic structure. Multi-university deployment requires dynamic institutional configuration.
2. **Mobile Client Format:** While fully responsive and PWA-enabled with client caching, dedicated native mobile application builds would provide enhanced hardware integration (e.g., biometric authentication).
3. **Manual Content Moderation:** Content reporting queues currently rely on human administrative review, which may present scaling constraints at high campus volumes.

8.4 Future Roadmap

- **Native Mobile Applications:** React Native clients targeting Android and iOS with native push notifications and QR code verification for physical handovers.
- **Multi-Campus Federation:** Multi-tenant architecture supporting inter-university resource exchange across regional universities in Bangladesh.

- **AI-Assisted Moderation:** Automated NLP content screening to assist moderators in identifying off-platform contact leakage and spam.
- **Institutional LMS Integration:** Single Sign-On (SSO) integration with university course portals for verified coursework assignments.

8.5 Final Remarks

PeerHive demonstrates that a purpose-built, institutional campus platform can solve the fragmentation, trust deficits, and payment hazards of informal student commerce. Anchored by the 51-table relational schema, 484+ automated tests, and live production deployment on Render and Vercel, the system establishes a secure, trustworthy digital foundation for university communities.

Bibliography

- [1] Microsoft Corporation, *ASP.NET Core 8 Documentation*, 2024. <https://learn.microsoft.com/aspnet/core/>
- [2] Microsoft Corporation, *Entity Framework Core 8 Documentation*, 2024. <https://learn.microsoft.com/ef/core/>
- [3] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024. <https://www.postgresql.org/docs/16/>
- [4] React Team, *React 18 Documentation*, 2024. <https://react.dev/>
- [5] Evan You, *Vite: Next Generation Frontend Tooling*, 2024. <https://vitejs.dev/>
- [6] Microsoft Corporation, *ASP.NET Core SignalR Documentation*, 2024. <https://learn.microsoft.com/aspnet/core/signalr/>
- [7] Jones, M., Bradley, J., and Sakimura, N., *RFC 7519 – JSON Web Token (JWT)*, Internet Engineering Task Force, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [8] OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, 2021. <https://owasp.org/Top10/>
- [9] bKash Limited, *bKash Developer Portal – Tokenized Checkout API*, 2024. <https://developer.bkash.com/>
- [10] Vercel Inc., *Vercel Documentation – Deployment Platform*, 2024. <https://vercel.com/docs>
- [11] Render Inc., *Render Documentation – Cloud Application Hosting*, 2024. <https://render.com/docs>
- [12] Bootstrap Team, *Bootstrap 5 Documentation*, 2024. <https://getbootstrap.com/docs/5.3/>
- [13] TanStack, *TanStack Query Documentation*, 2024. <https://tanstack.com/query/>
- [14] Jeremy Skinner, *FluentValidation Documentation*, 2024. <https://docs.fluentvalidation.net/>
- [15] Serilog Contributors, *Serilog – .NET Logging Framework*, 2024. <https://serilog.net/>
- [16] Cloudinary Ltd., *Cloudinary – Image and Video API*, 2024. <https://cloudinary.com/documentation>
- [17] Neon Inc., *Neon – Serverless Postgres*, 2024. <https://neon.tech/docs/>
- [18] GitHub Inc., *GitHub Actions Documentation*, 2024. <https://docs.github.com/en/actions>
- [19] Microsoft Corporation, *Visual Studio 2022 Documentation*, 2024. <https://learn.microsoft.com/visualstudio/>
- [20] Microsoft Corporation, *C# 12 Language Specification*, 2024. <https://learn.microsoft.com/dotnet/csharp/>